

# Dynamic memory allocation

**C programming · CM10 · 90 minutes**

ISTIC · University of Rennes 2026-2027



Louis Ledoux

**\$ whoami**

**Not a C expert ... : ' )**

# Louis Ledoux

## Associate Professor

University of Rennes · ISTIC

IRISA / Inria · Taran

► Arithmetic circuits: **MLIR (LLVM)**  
/ **CIRCT**.

► **Systolic arrays** and small  
floating-point formats.

Previously: Inria / INSA Lyon,  
Emeraude.

PhD: **BSC** / **UPC**.

[bynaryman.github.io](https://bynaryman.github.io)



**MareNostrum 4** · BSC,  
Barcelona

## Dynamic memory allocation on the heap

We open an image and discover its dimensions. Where do we store its pixels?

1. Make an independent copy.
2. Store values and find memory errors.
3. Organise arrays, strings and matrices.
4. Group data with user-defined types.

# 1. Copy a string

Can we change the copy and keep the original?

# A copy we can edit

Keep **hello** in the original.

Change its copy to **Hello**.

demos/allocation/01-copy-alias.c

Terminal: make run FILE=01-copy-alias.c

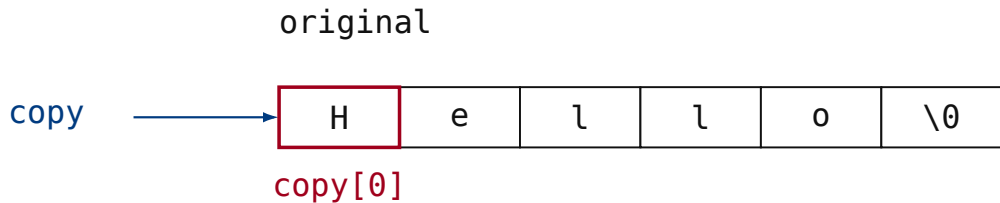
# Did we copy the characters?

```
char original[] = "hello";  
char *copy = original;
```

```
copy[0] = 'H';  
printf("original: %s\n", original);  
printf("copy:      %s\n", copy);
```

```
original: Hello  
copy:      Hello
```

# One array, two ways to reach it



`char *copy = original;` copies the **address**.

# Separate storage

How many bytes does the copy need?

```
char *copy = malloc(strlen(original));  
if (copy == NULL)  
    return EXIT_FAILURE;
```

malloc requests bytes. Check its result **before using it**.

# Copying the characters

What should the two lines print now?

```
strcpy(copy, original);  
copy[0] = 'H';  
printf("original: %s\n", original);  
printf("copy:      %s\n", copy);
```

strcpy copies the characters **and the final '\0'**.

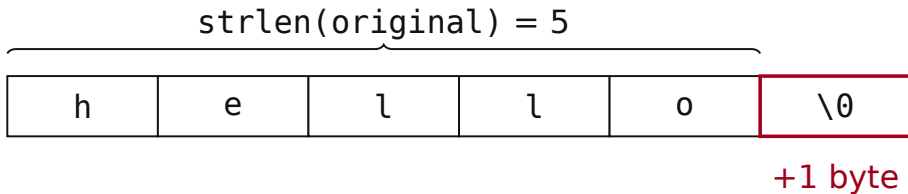
# It prints. Is the copy valid?

```
make valgrind FILE=02-copy-short.c
```

**Invalid write** in strcpy

The destination block has **5 bytes**.

# Five letters, six bytes



```
char *copy = malloc(strlen(original) + 1);
```

# The copy works. What remains?

```
make valgrind FILE=03-copy-leak.c
```

**6 bytes definitely lost**

We allocated the copy but never released it.

# Release after the last use

The original and its copy now print correctly.

```
printf("%s\n", original);  
printf("%s\n", copy);  
free(copy);
```

`free(copy)` releases the allocated block.

**No leak**

Valgrind reports zero errors.

# Two independent strings

original

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| h | e | l | l | o | \0 |
|---|---|---|---|---|----|

copy



|   |   |   |   |   |    |
|---|---|---|---|---|----|
| H | e | l | l | o | \0 |
|---|---|---|---|---|----|

separate allocation

**Before free:** changing `copy[0]` leaves `original[0]` unchanged.

## 2. Arrays and memory errors

We copied existing characters. Now we create an array of values.  
How many **bytes**? What values are there **before we write**?

# How much space for integers?

We need space for **100 values**, initially set to zero.

```
int *p = malloc(count * sizeof(int));
if (p == NULL) {
    fputs("Allocation failed\n", stderr);
    return EXIT_FAILURE;
}
```

demos/allocation/05-integers.c

Terminal: make run FILE=05-integers.c

# Count elements, request bytes

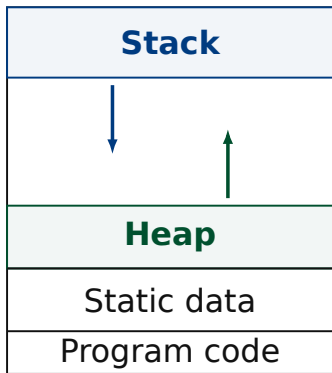
```
void *malloc(size_t size);    // From <stdlib.h>  
int *p = malloc(100 * sizeof(int));
```

- ▶ `size_t` is unsigned, with at least **16 value bits**.
- ▶ `sizeof` is an **operator** giving a size in bytes.
- ▶ `sizeof(int)` is the size of one `int`, not the pointer.

Sizes depend on the platform. In C, the `void *` result needs no cast.

# Where does the program keep its data?

0xffff



0x0000

count and pointer p  
Automatic locals

Room to grow  
Opposing directions

p[0] to p[count-1]  
Allocated integers

Globals and static

Illustrative 16-bit address space; real address widths and layouts vary.

# What will the array contain?

malloc succeeded. We removed the loop that wrote zeros.

```
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

## Uninitialised value

Valgrind reports that  
printf uses values we  
never wrote.

malloc reserves storage; it **does not initialise the integers**.

# Write before reading

Restore the loop **before** printf.

```
for (int i = 0; i < count; ++i)
    p[i] = 0;

printf("first=%d, last=%d\n",
      p[0], p[count - 1]);
```

Every element now has a value before it is read.

**Zero errors**  
Rerun Valgrind to  
check this execution.

# Four integers: which write is outside?

`count = 4`. The allocation holds `p[0]` through `p[3]`.

```
for (int i = 0; i <= count; ++i)
    p[i] = 0;
```

## Invalid write

`<=` also allows `i = 4`.

There is **no element** `p[4]`  
in this allocation.

Use **`i < count`**: four writes, with indices **0, 1, 2, 3**.

# Can we read after free?

p points to four integers initialised to zero.

```
free(p);  
  
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

After free(p), **p is a dangling pointer.**

## Invalid read

Valgrind points to printf:  
the block was **already  
freed.**

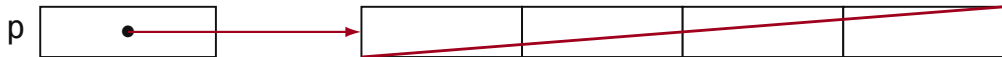
# Dangling pointer

A pointer to an object **whose lifetime has ended**.

`free(p)` releases the block; it **does not set p to NULL**.

Pointer

Released block



Do not access these elements

**Repair: finish reading before releasing the block.**

```
printf("first=%d, last=%d\n", p[0], p[count - 1]);  
free(p);
```

# Can we return this array?

Move array creation into a function. Can its caller use the array?

```
int *make_values(void) {  
    int values[4] = {0};  
    return values;  
}
```

```
// In the caller:  
int *p = make_values();
```

**During the call**  
values exists and  
contains four zeros.

**After the return**  
values no longer exists. **p**  
**is dangling**, without any  
free.

Returning an address **does not keep a local object alive.**

# When does the storage end?

The pointer and the object it refers to have **separate lifetimes**.

| Storage                 | Lifetime ends                 |
|-------------------------|-------------------------------|
| Ordinary local array    | When its block ends           |
| Block from malloc       | When it is released with free |
| Global or static object | When the program ends         |

Each recursive call has its own locals. Too many calls can **exhaust the stack**.

# Does setting p to NULL free the block?

The block is still allocated. What if we erase its **only address**?

```
p = NULL;  
free(p);
```

free(NULL) does nothing.  
We lost the address: a  
**memory leak.**

Pointer

|   |      |
|---|------|
| p | NULL |
|---|------|

Allocated block

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
|---|---|---|---|

Address lost; block still allocated

# Who releases an unreachable block?

## C: explicit responsibility

For every successful allocation, decide who will release it and when.  
Repeated leaks consume more memory.

## Java: garbage collection

The runtime can reclaim unreachable objects automatically.

# Four functions in `<stdlib.h>`

We allocated, filled and released a block. We can also request zeros or resize it.

| Function                         | Purpose                             |
|----------------------------------|-------------------------------------|
| <code>malloc(bytes)</code>       | Allocate uninitialised storage      |
| <code>calloc(count, size)</code> | Allocate storage with all bits zero |
| <code>realloc(p, bytes)</code>   | Resize an allocation; it may move   |
| <code>free(p)</code>             | Release an allocation               |

# The free contract

```
void free(void *pointer);    // Declared in <stdlib.h>
```

- ▶ Pass the allocation's **start address**, once.
- ▶ `free(NULL)` does nothing; `free` returns no value.
- ▶ An invalid or repeated release is **undefined behaviour**.

# Checkpoint · questions 1-5

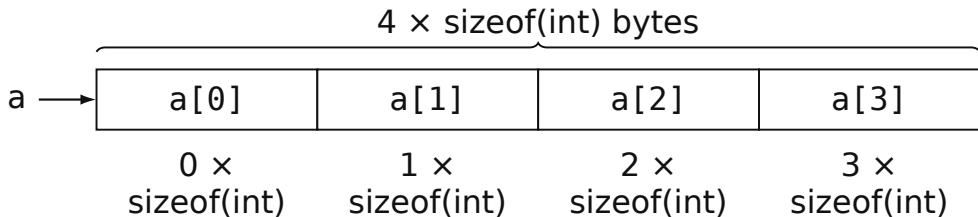
Answer questions **1-5**, then discuss your reasoning.  
Open the allocation quiz.

### 3. Arrays, strings and matrices

An image is a grid of values. How can we organise its rows in memory?

# A one-dimensional integer array

```
int *a = malloc(4 * sizeof(int));
```



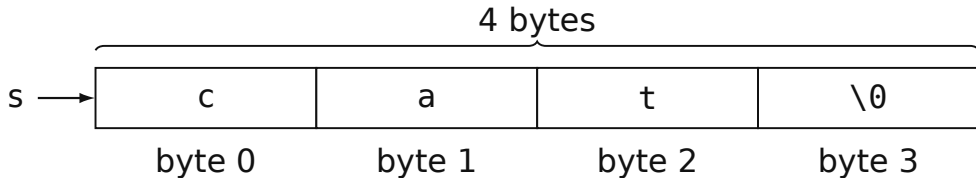
Use  $a[i]$ , equivalently  $*(a + i)$ , for  $0 \leq i < 4$ .

demos/allocation/10-array.c

Terminal: make run FILE=10-array.c

# A character array can hold a string

```
char *s = malloc(4 * sizeof(char));
```



Three letters need **four bytes**, including `'\0'`.

`demos/allocation/11-string.c`

Terminal: `make run FILE=11-string.c`

# Assigning a pointer is not copying text

Predict the values of p and s, and the fate of the allocation.

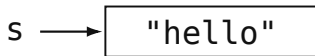
```
char *s = malloc(100);  
if (!s) return EXIT_FAILURE;  
s = "hello";  
char *p = s;  
s = "bye";
```

demos/allocation/12-string-assignment.c

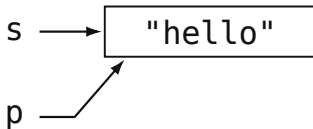
Terminal: make run FILE=12-string-assignment.c

# Follow the three assignments

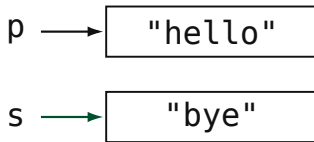
`s = "hello"`



`p = s`



`s = "bye"`



Original heap block: 100 bytes, now unreachable

The literals are separate from the lost heap block. Neither pointer can now be passed to `free`.

# Copy characters into owned storage

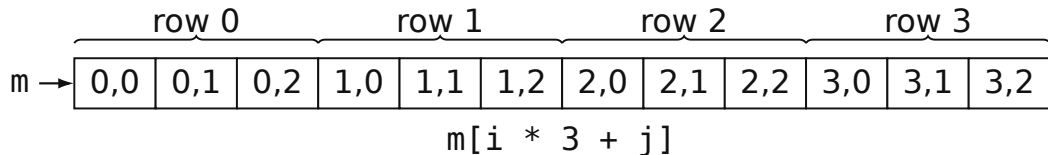
```
char *s = malloc(6);  
if (!s) return EXIT_FAILURE;  
strcpy(s, "hello"); // Include <string.h>.  
printf("%s\n", s);  
free(s);
```

Alternatively, just reference a literal

Use `const char *s = "hello";` when no writable copy is needed.

# A 2D array in one block

```
int *m = malloc(4 * 3 * sizeof(int));  
/* After checking m: element (i, j) is m[i * 3 + j]. */
```



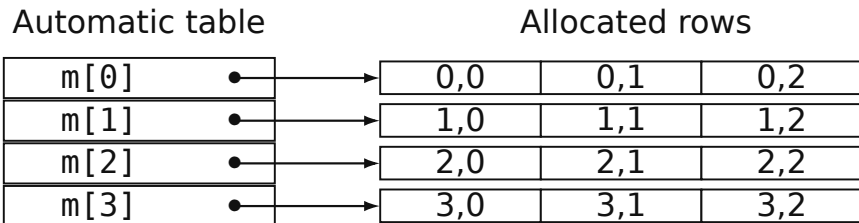
One allocation, one **free(m)**.

demos/allocation/13-matrix-flat.c

Terminal: make run FILE=13-matrix-flat.c

# An array of row pointers

```
int *m[4]; // Automatic table of four pointers.  
for (int i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof(int));
```



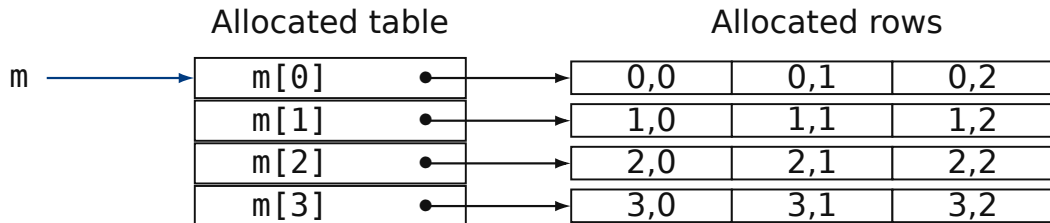
Check each row. Use `m[i][j]`; free each row, **not** `m`.

`demos/allocation/14-matrix-rows.c`

Terminal: `make run FILE=14-matrix-rows.c`

# Allocate the pointer table too

```
int **m = malloc(4 * sizeof(int *));  
for (int i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof(int));
```



Check `m` first, then each row. Free **rows first**, then `free(m)`.

`demos/allocation/15-matrix-dynamic.c`

Terminal: `make run FILE=15-matrix-dynamic.c`

# Checkpoint · questions 6-8

Answer questions **6-8**, then discuss your reasoning.  
Open the allocation quiz.

## 4. User-defined types

Group data and access its members.

We want to keep a street name and a house number in one object.

## Three tools

- ▶ struct: group the street and number.
- ▶ enum: name integer constants.
- ▶ typedef: give a type another name.

`demos/allocation/16-types.c`

Terminal: `make run FILE=16-types.c`

# Enumerations

```
enum day { mon, tue, wed, thu, fri, sat, sun };  
enum day today;  
today = wed;
```

By default, values start at **0** and increase by **1**.

## Named integer constants

mon is 0; wed is 2; sun is 6.

# A structure groups several members

```
struct address {  
    char street[100];  
    int number;  
};
```

## A type definition

Members may be integers, floating-point values, arrays, pointers or other structures. No address object exists yet.

# Define a type, then create objects

```
struct address home;  
struct address office;
```

These definitions allocate **two distinct objects** of the same type.

## Local objects need initialisation

Ordinary local members have indeterminate values until initialised or assigned. The type definition alone sets no values.

# Initialising a structure

```
struct address home = { "Paul Bert", 12 };
```

Or combine the type and object definitions:

```
struct address {  
    char street[100];  
    int number;  
} home = { "Paul Bert", 12 };
```

# Same members, different order

```
struct grouped {  
    int id1, id2;  
    char c1, c2;  
    float ratio;  
};  
struct interleaved {  
    int id1; char c1;  
    int id2; char c2;  
    float ratio;  
};
```

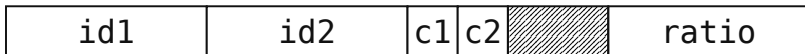
## Padding

The implementation may insert unused bytes between members and at the end to satisfy alignment requirements.

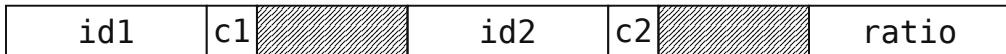
- ▶ Members appear in declaration order.
- ▶ `sizeof(struct T)` includes padding.
- ▶ Layout and alignment depend on the implementation.

# See where the padding goes

Grouped: 16 bytes



Interleaved: 20 bytes



Here: 4-byte int/float, alignment 4; 1-byte char.

**16 vs 20 bytes** in this layout. Measure with `sizeof` and `offsetof`.

`demos/allocation/17-padding.c`

Terminal: `make run FILE=17-padding.c`

# Access members through an object

```
struct address home = { "Paul Bert", 12 };  
strcpy(home.street, "Rue de Paris");  
home.number = 14;
```

## The dot operator

Use `.` with a structure object.

`strcpy` needs `<string.h>` and enough space for the terminator.

# Access members through a pointer

```
struct address *p = &home;  
strcpy(p->street, "Rue de Paris");  
p->number = 16;
```

## The arrow operator

Use `->` with a pointer. `p->number` is the same member as `(*p).number`.

Both forms above modify **home**.

# typedef: a name for a type

```
typedef int LENGTH;  
typedef enum { FALSE, TRUE } BOOLEAN;  
typedef struct date DATE;
```

## Alias

A typedef introduces a type name, not an object or a distinct underlying type. Then write `LENGTH width;`.

# Combine a date and a day of the week

```
struct date { short day, month; int year; };  
typedef enum { mon, tue, wed, thu, fri, sat, sun } DAY;  
typedef struct { DAY day; struct date date; } A_DATE;
```

## A nested structure

A\_DATE contains a DAY and a complete struct date.

# Access nested members

```
A_DATE today = { wed, { 3, 9, 2014 } };  
today.day = wed;  
today.date.year = 2014;  
A_DATE *p = &today;  
p->day = today.day;  
p->date.year = 2026;
```

p->date.year reaches the nested member of **the same object**.

demos/allocation/18-nested.c

Terminal: make run FILE=18-nested.c

# Checkpoint · questions 9-12

Answer questions **9-12**, then discuss your reasoning.  
Open the allocation quiz.